Mobile AI Project

FocusFlow+

AI-guided focus sessions based on task, energy, and available time

Student	Jaouher Chouchane
Profile	IT/BA Junior Student
Instructor	Prof. Najet Boughanmi

Technology Stack

Android	Kotlin	Jetpack Compose	Gemini API
Retrofit/OkHttp	Room Database	MVVM	Git/GitHub

Academic Year 2025–2026

Abstract

FocusFlow+ is a native Android mobile application that helps users transform a task into a realistic focus session. Instead of using a fixed timer for every situation, the application collects the task description, current energy level, and available time. These inputs are sent to the Gemini API, which generates a structured recommendation containing focus duration, break duration, session structure, health tips, and a short motivational message.

The application was implemented using Kotlin, Jetpack Compose, Retrofit, and Room Database. It follows an MVVM-inspired architecture where the UI is separated from state management, repositories, AI communication, and local persistence. The final MVP covers the full workflow: user input, AI-generated plan, timer execution, session summary, and saved history.

Keywords: Android, Kotlin, Jetpack Compose, Gemini API, Room Database, MVVM, productivity, focus timer.

Contents

1 Introduction

1.1 Project Context

Students and busy users often know what they need to do, but they struggle to transform that intention into a concrete work session. A task can feel too large, the available time may be limited, and the user's energy level may not match the effort required. In practice, the challenge is not only completing the task; it is also deciding how to start, how long to focus, when to take a break, and how to keep the plan realistic.

Traditional focus timer applications often apply a fixed pattern such as 25 minutes of work and 5 minutes of break. This is useful for basic timeboxing, but it ignores the user's current context. FocusFlow+ addresses this limitation by adding an AI planning step before the timer starts.

1.2 Objectives

The objective of FocusFlow+ is to build a mobile AI assistant that:

- collects task description, energy level, and available time;
- uses Gemini to generate a personalized focus plan;
- guides the user through a focus/break timer;
- saves planned and completed sessions locally;
- allows the user to review summaries and history.

1.3 Technology Overview

Table 1: Main technologies used in the project

Technology	Role in the application
Kotlin	Main programming language for Android logic.
Jetpack Compose	Declarative UI toolkit used to build mobile screens and components.
Gemini API	External AI service used to generate personalized focus plans.
Retrofit/OkHttp	Networking layer used to call the Gemini REST endpoint.
Room Database	Local persistence layer used to store planned and completed sessions.
MVVM-inspired structure	Separates screens, ViewModels, repositories, and data models.

2 Problem Analysis

Real-world pain point

"I know what I need to do, but I do not know how to turn it into a realistic work session."

This situation is common for students, junior professionals, and anyone trying to complete a task under time and energy constraints. The user may waste effort deciding how to divide the session instead of starting the work itself.

Table 2: Main problem causes

Cause	Description	Impact
Planning overload	The user must decide where to start, how long to focus, and when to rest.	Delayed start and lower motivation.
Fixed timers	Static timer patterns ignore task type, available time, and energy level.	The plan may be unrealistic or unsuitable.
Energy mismatch	Tired users and energized users receive the same timer structure.	Sessions can feel too difficult or too light.
No progress memory	Many simple timers do not keep useful local history.	Users cannot easily review consistency or past work.

2.1 Target Users

The target users are people who need a lightweight way to move from intention to action: students preparing revisions or assignments, busy users planning a focused work block, and users who prefer a simple assistant instead of a complex productivity system.

3 Proposed Solution

FocusFlow+ solves the problem by using AI as a planning layer before the timer. The user provides three inputs: task description, energy level, and available time. Gemini uses this context to generate a practical recommendation, and the app converts that recommendation into a guided session.

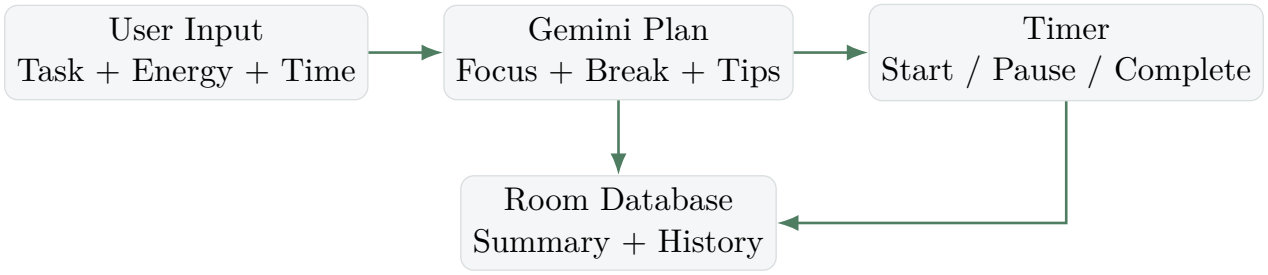


Figure 1: Solution workflow

3.1 Value of the Solution

The application reduces uncertainty before starting. It considers more than only time: the task, the current energy level, the available duration, break needs, wellness guidance, and motivation. This makes the focus session more realistic and easier to start.

MVP scope

The implemented MVP includes AI plan generation, timer execution, local storage, summary screen, history screen, input validation, and error display. It does not yet include authentication, cloud synchronization, social features, or a backend AI proxy.

4 Requirements Analysis

4.1 Functional Requirements

Table 3: Functional requirements

ID	Requirement	Description
FR1	Enter task	User writes the task they want to work on.
FR2	Select energy	User selects current energy level.
FR3	Select time	User selects available session time.
FR4	Generate plan	App calls Gemini and receives a structured recommendation.
FR5	View recommendation	App displays focus time, break time, tips, and motivation.
FR6	Control timer	User can start, pause, reset, go to break, and complete.
FR7	Save session	App stores planned and completed sessions locally.
FR8	View summary/history	User reviews the current result and previous sessions.

4.2 Non-Functional Requirements

Table 4: Non-functional requirements

Attribute	Requirement
Usability	Clear mobile-first interface with readable cards and obvious actions.
Performance	Lightweight local storage and one Gemini call per generated plan.
Reliability	Input validation, clear error messages, and persistent local history.
Maintainability	Separation between UI, ViewModels, repositories, services, and models.
Security	API key excluded from Git; production version should use a backend proxy.
Extensibility	Structure can support profiles, reminders, analytics, and feedback later.

5 System Design

5.1 Use Case Diagram

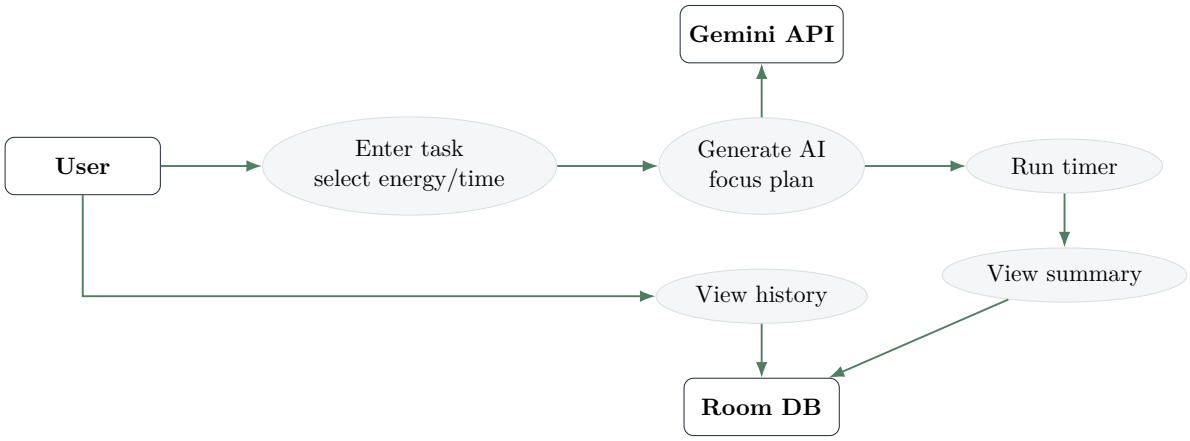


Figure 2: Use case diagram

5.2 Architecture

The project uses a layered MVVM-inspired design. Screens display UI, ViewModels manage state and actions, repositories coordinate data, and data sources represent Gemini and Room.

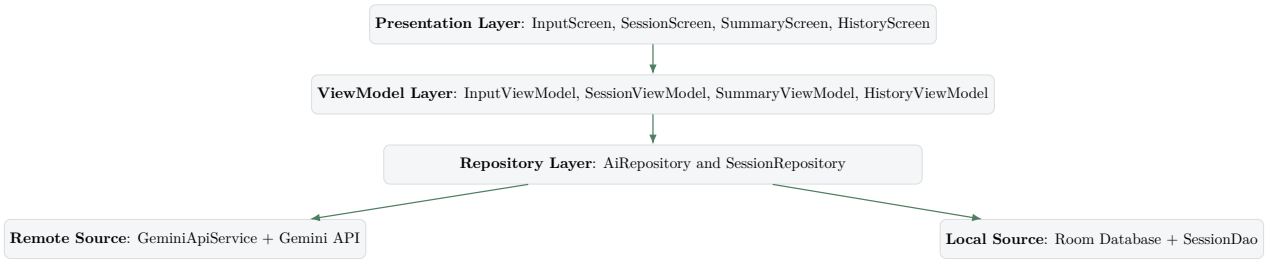


Figure 3: System architecture

5.3 Database Design

The Room database stores planned and completed focus sessions. This ensures that the session history remains available after the application is closed.

Table 5: SessionEntity database design

Field	Type	Purpose
id	Long	Primary key, auto-generated identifier.
taskDescription	String	Task entered by the user.
energyLevel	Int	User energy level used for personalization.
availableMinutes	Int	Total available time selected by the user.
focusMinutes	Int	Focus duration returned by Gemini.
breakMinutes	Int	Break duration returned by Gemini.
completedAt	Long	Timestamp used for ordering and history.
wasCompleted	Boolean	Distinguishes planned and completed sessions.
aiRecommendation	String	Stores the AI recommendation text.

5.4 Class Diagram

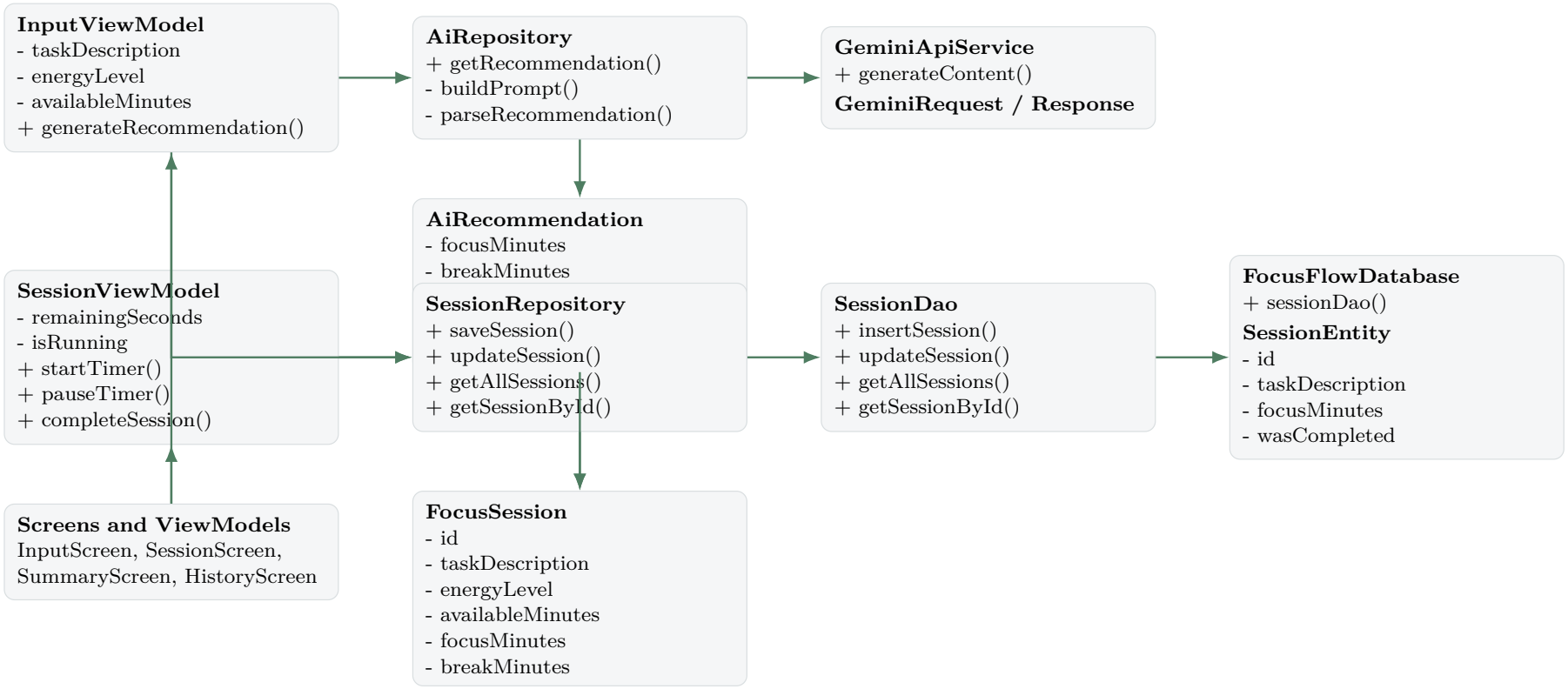


Figure 4: Class diagram grouped by responsibility

6 Implementation

6.1 Development Environment

The project was developed in Android Studio using Kotlin. The user interface was built with Jetpack Compose, the network layer with Retrofit and OkHttp, the local database with Room, and asynchronous operations with Kotlin coroutines and Flow.

6.2 User Input and Gemini Integration

The planner screen collects the three values required for personalization. The ViewModel validates the input, calls the AI repository, and navigates to the session workflow when a recommendation is available.

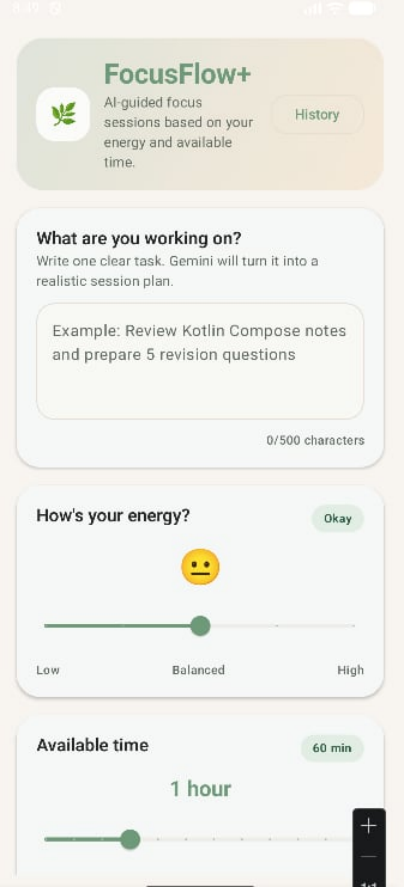


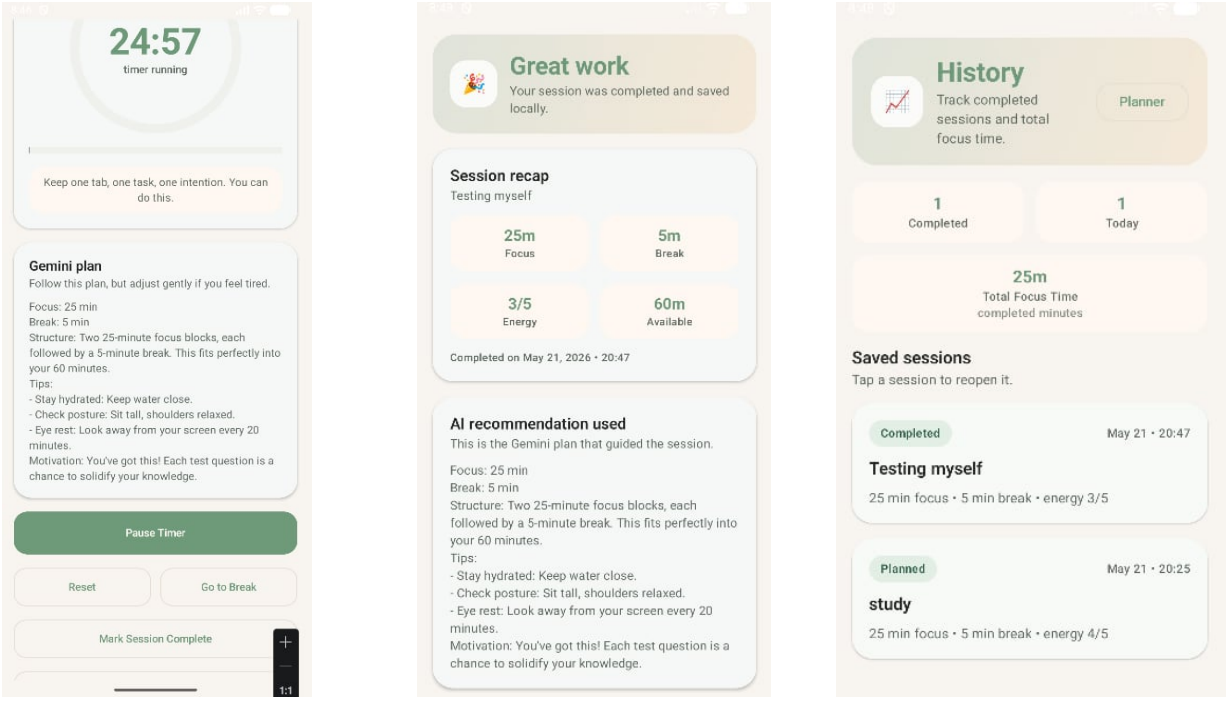
Figure 5: Planner screen collecting task, energy, and available time

Listing 1: Gemini Retrofit service

```
interface GeminiApiService {
    @POST("v1beta/models/{model}:generateContent")
    suspend fun generateContent(
        @Path("model") model: String,
        @Query("key") apiKey: String,
        @Body request: GeminiRequest
    ): GeminiResponse
}
```

The response is parsed into an `AiRecommendation`, which is then used by the timer workflow. The app reads the API key from `local.properties`, which is excluded from GitHub for local development.

6.3 Timer, Summary, and History



(a) Timer (b) Summary (c) History

Figure 6: Main implemented screens after AI recommendation

When the user completes a session, the app updates the session state and saves it through Room. The summary screen confirms the result, and the history screen lists planned and completed sessions with basic statistics.

7 Testing and Validation

Testing focused on the complete MVP workflow and common error conditions. The application was tested manually on an Android emulator.

Table 6: Manual validation checklist

ID	Test case	Expected result
T1	Generate without task	App displays a validation error.
T2	Missing API key	App displays a clear configuration error.
T3	Real Gemini generation	App returns focus/break time, structure, tips, and motivation.
T4	Timer controls	Start, pause, reset, break, and complete actions update the state.
T5	Session completion	App navigates to summary and marks the session completed.
T6	Local history	Planned and completed sessions remain visible in history.
T7	App restart	Saved sessions remain available because they are stored locally.

Validated workflow

Input → Gemini plan → Timer → Summary → History

8 Limitations and Future Improvements

8.1 Current Limitations

The MVP is functional, but several limitations remain. The Gemini call is made from the Android client for local academic development. Network availability can affect AI generation. Automated tests are not yet implemented. The application does not include user accounts, cloud synchronization, or long-term analytics.

8.2 Future Improvements

- **Backend AI proxy:** move the Gemini call to a backend to protect the API key in production.
- **User profiles:** store preferences such as preferred session length and goals.
- **Adaptive feedback:** let users rate sessions and use feedback to improve future prompts.
- **Notifications:** add reminders and scheduled focus sessions.
- **Analytics:** show weekly focus time, streaks, and progress trends.
- **Automated testing:** add ViewModel, Room, and Compose UI tests.

9 Conclusion

FocusFlow+ demonstrates how AI can improve a simple productivity timer by making it context-aware. Instead of asking the user to manually decide how to divide time, the application collects task context, energy level, and available time, then uses Gemini to generate a realistic focus plan. The generated plan becomes part of the timer workflow and is saved locally for later review.

From a technical point of view, the project combines modern Android development with AI integration and local persistence. Jetpack Compose provides the interface, ViewModels manage state, Retrofit communicates with Gemini, and Room stores sessions locally. The final result is a complete MVP that helps users move from uncertainty to action.

References

References

[1] Android Developers. *Jetpack Compose documentation*. https://developer.android.com/compose

[2] Android Developers. *Room persistence library*. https://developer.android.com/training/data-storage/room

[3] Google AI for Developers. *Gemini API documentation*. https://ai.google.dev/gemini-api/docs

[4] Square. *Retrofit documentation*. https://square.github.io/retrofit/

[5] JetBrains. *Kotlin documentation*. https://kotlinlang.org/docs/home.html